UNIT-3

Processor

Revisiting Clock Methodology

- **Clocking Basics**
- Clock defines **state update boundary**
- Sequential elements (flip-flops, latches) sample at clock edge
- Clock period determines **max operating frequency**

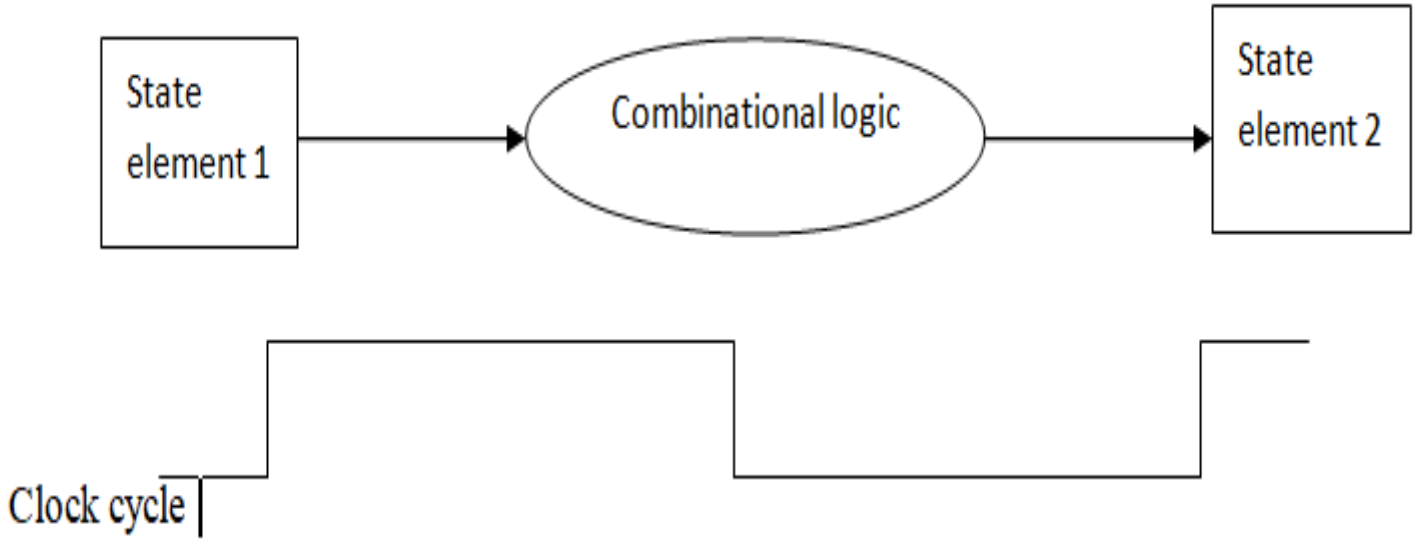
Modern Clocking Concerns

- **1. Clock Skew**
- Difference in clock arrival time at different flip-flops.
- Positive skew → setup violation
- Negative skew → hold violation
- Mitigation:
- Balanced clock tree (H-tree)
- Clock mesh
- Careful CTS (Clock Tree Synthesis)
- **2. Clock Jitter**
- Short-term clock variation.
- Sources:
- PLL noise
- Power supply noise
- Crosstalk

3.Clock Distribution Structure

- Clock tree
- H-tree (balanced)
- Clock mesh (high-performance CPUs)

Clock diagram



Amdahl's Law

- It is named after computer scientist Gene Amdahl(a computer architect from IBM and Amdahl corporation) and was presented at the AFIPS Spring Joint Computer Conference in 1967. It is also known as ***Amdahl's argument***.
- It is a formula that gives the theoretical speedup in latency of the execution of a task at a fixed workload that can be expected of a system whose resources are improved

Terms used in ***Amdahl's Law***

- **Speedup**
- **Fraction enhanced**
- **Speedup enhanced**

$$\text{Overall Speedup} = \frac{\text{Old execution time}}{\text{New execution time}}$$

$$= \frac{1}{\left((1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}} \right)}$$

The formula for Amdahl's law is:

- $S = 1 / (1 – P + (P / N))$
- Where:
- S is the speedup of the system
  P is the proportion of the system that can be improved
  N is the number of processors in the system
- For example, if a system has a single bottleneck that occupies 20% of the total execution time, and we add 4 more processors to the system, the speedup would be:
- $S = 1 / (1 – 0.2 + (0.2 / 5))$
  $S = 1 / (0.8 + 0.04)$
  $S = 1 / 0.84$
  $S = 1.19$

- Disadvantages of Amdahl’s law

1. The portion of the program that cannot be parallelized is fixed, which may not be the case in practice
2. All processors have the same performance characteristics, which may not be the case in practice

Example

$$Speedup = \frac{1}{(1-f) + \frac{f}{S}}$$

Where:

- f = fraction improved
- S = speedup of improved part

Example

If 40% of program improved by 4×:

$$Speedup = \frac{1}{0.6 + 0.4/4} = \frac{1}{0.7} = 1.43$$

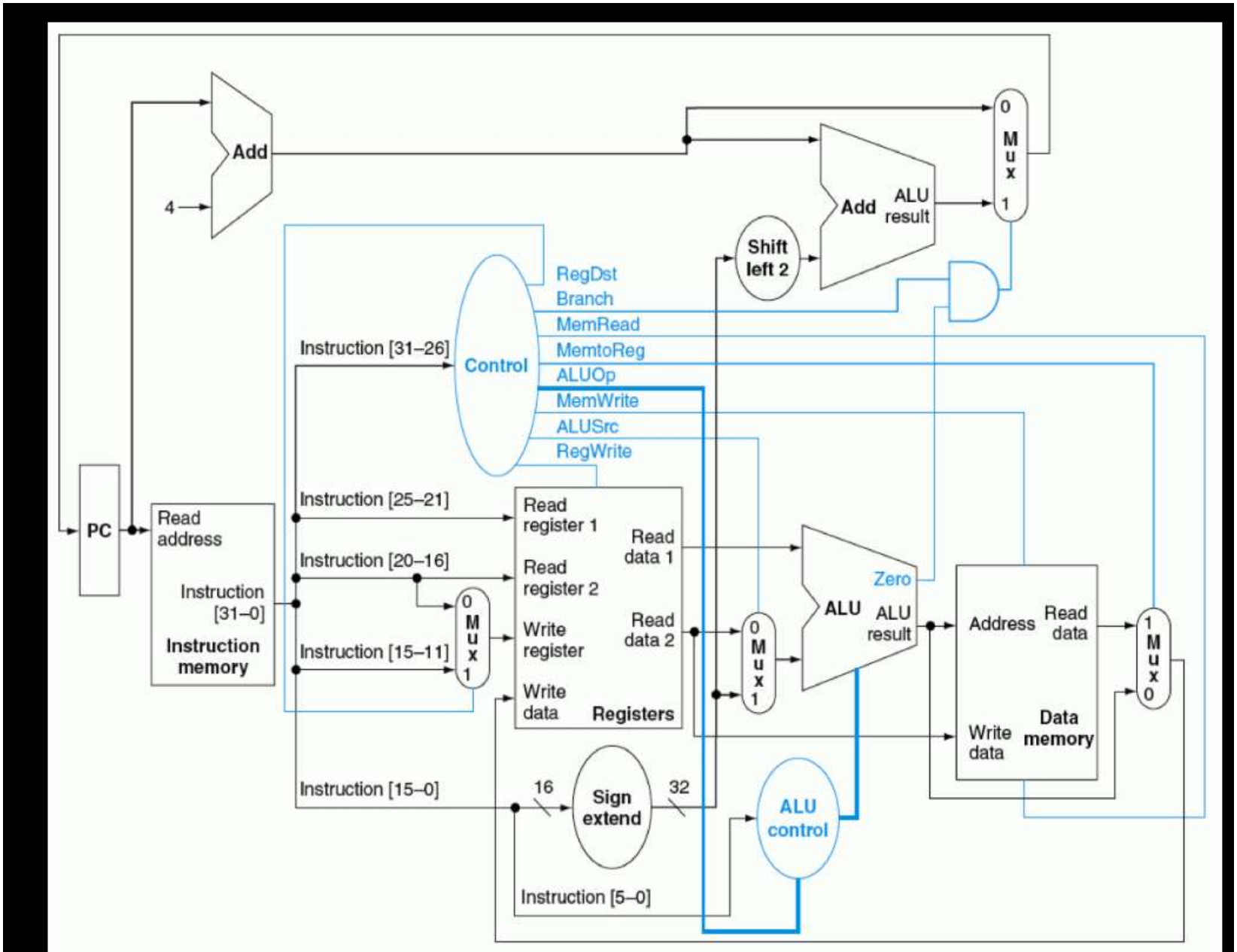
BUILDING A DATA PATH AND CONTROL

- **Datapath**
- It is a collection of function units organized in a manner to execute each class of instruction.
- **Datapath elements**
- A unit used to operate on or hold data within a processor is called datapath element.
- In the MIPS implementation, the datapath elements include the instruction and data memories, the register file, the ALU, and adders.
- **How to build a datapath:**
- Datapath design begins in examining the major components required to execute each class of MIPS instructions.
- data path elements each instruction needs and also their control signals.

Datapath

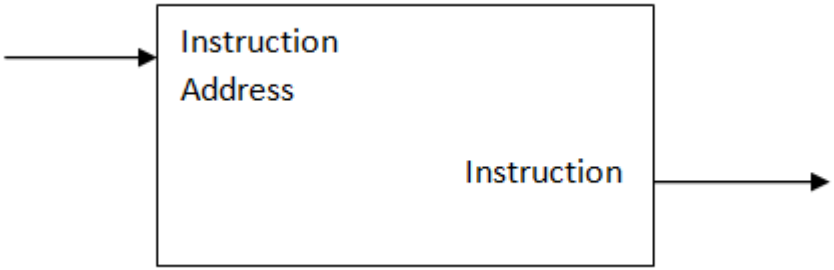
- Hardware that performs operations:
- ALU
- Registers
- Memory
- MUXes
- **Control Unit**
- Generates control signals:
- ALUOp
- RegWrite
- MemRead/Write
- Branch

BUILDING A DATA PATH

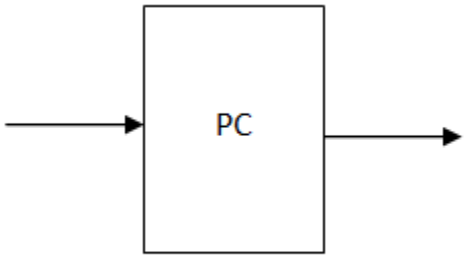


stages used in building datapath

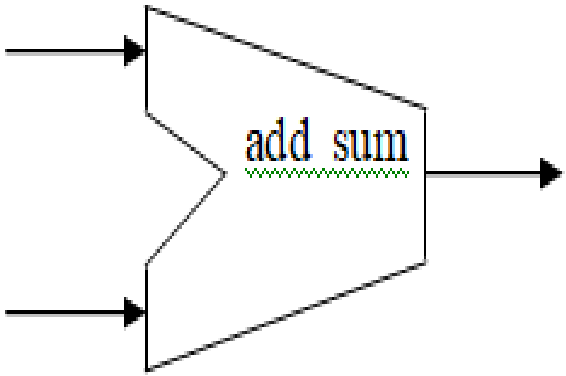
- Stage: 1 [Datapath to fetch instruction and increment PC]
- Instruction memory - a memory unit to store the instructions of a program and supply instructions given an address



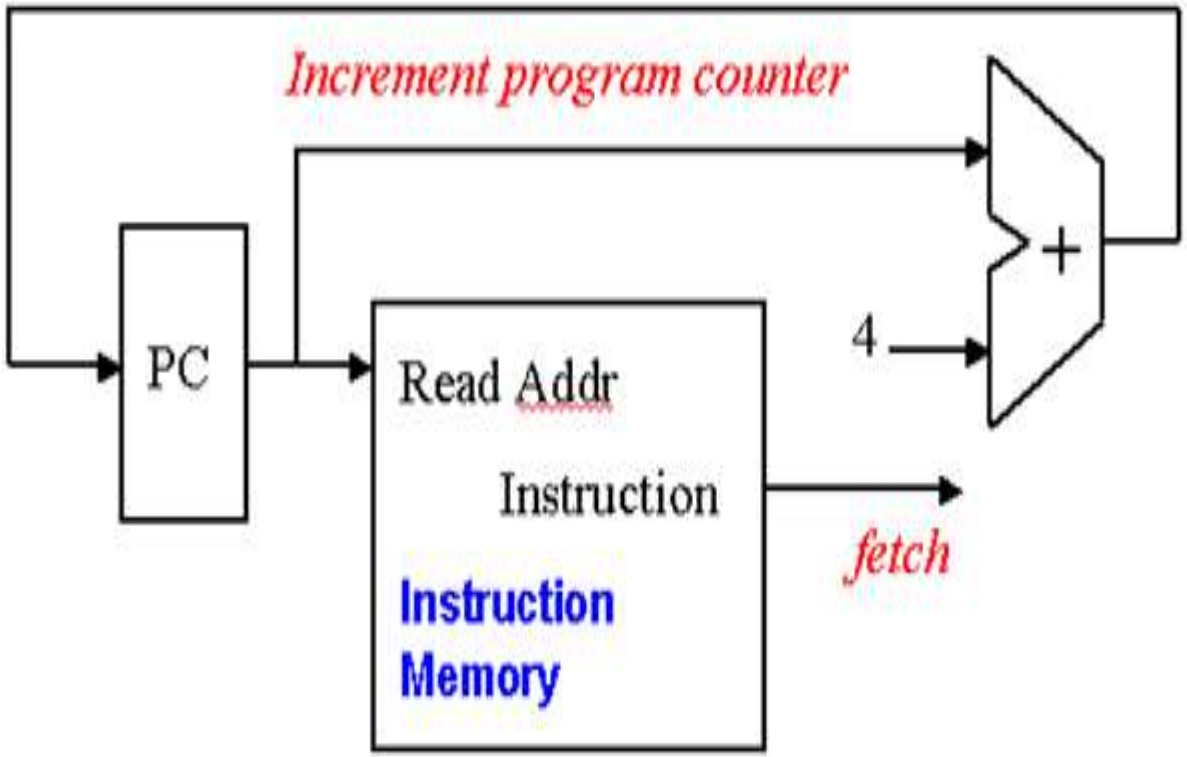
- Program counter
- The register containing the address of the instruction in the program being executed is called program counter



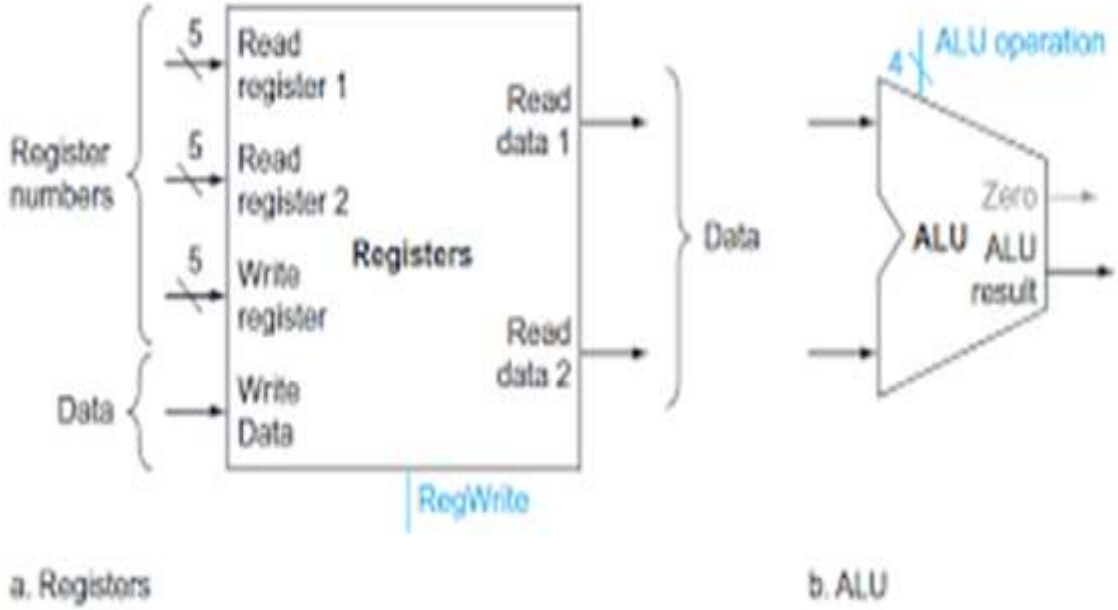
- Adder
- Adder is used to increment the PC to the address of the next instruction.
- The adder is an ALU wired to always add its two 32-bit inputs and place the sum on its output.



Combined all three elements into single stage

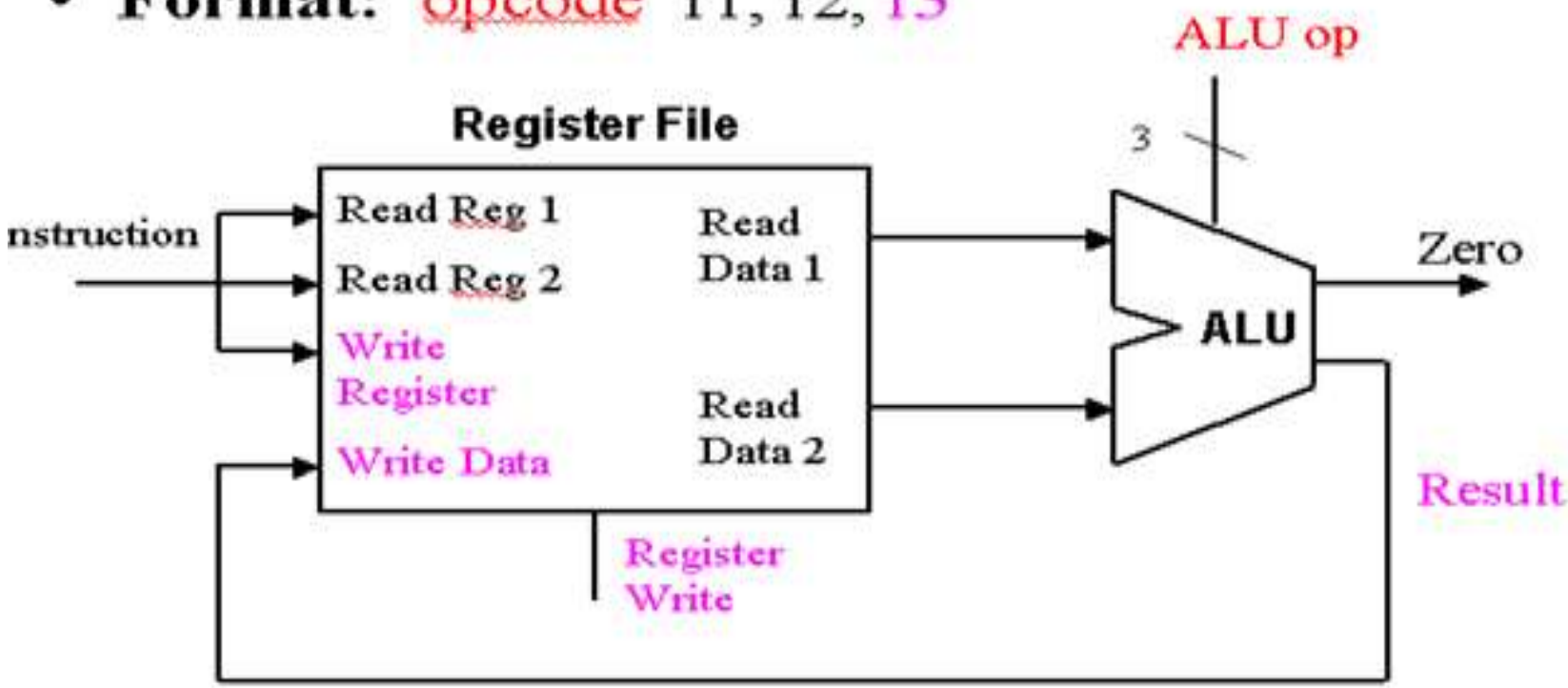


Stage:2[Datapath segment for multiport register file and the ALU] Register File:

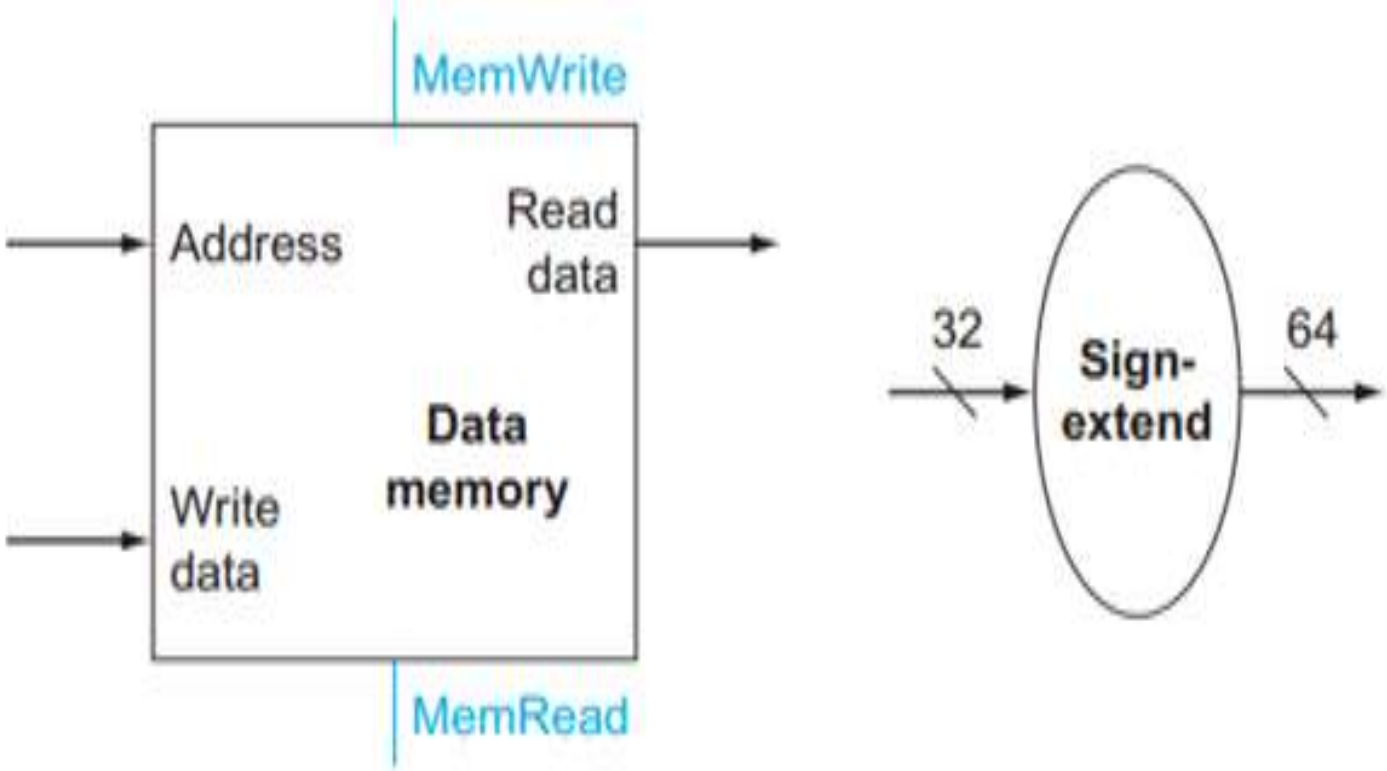


Combined two elements into single stage

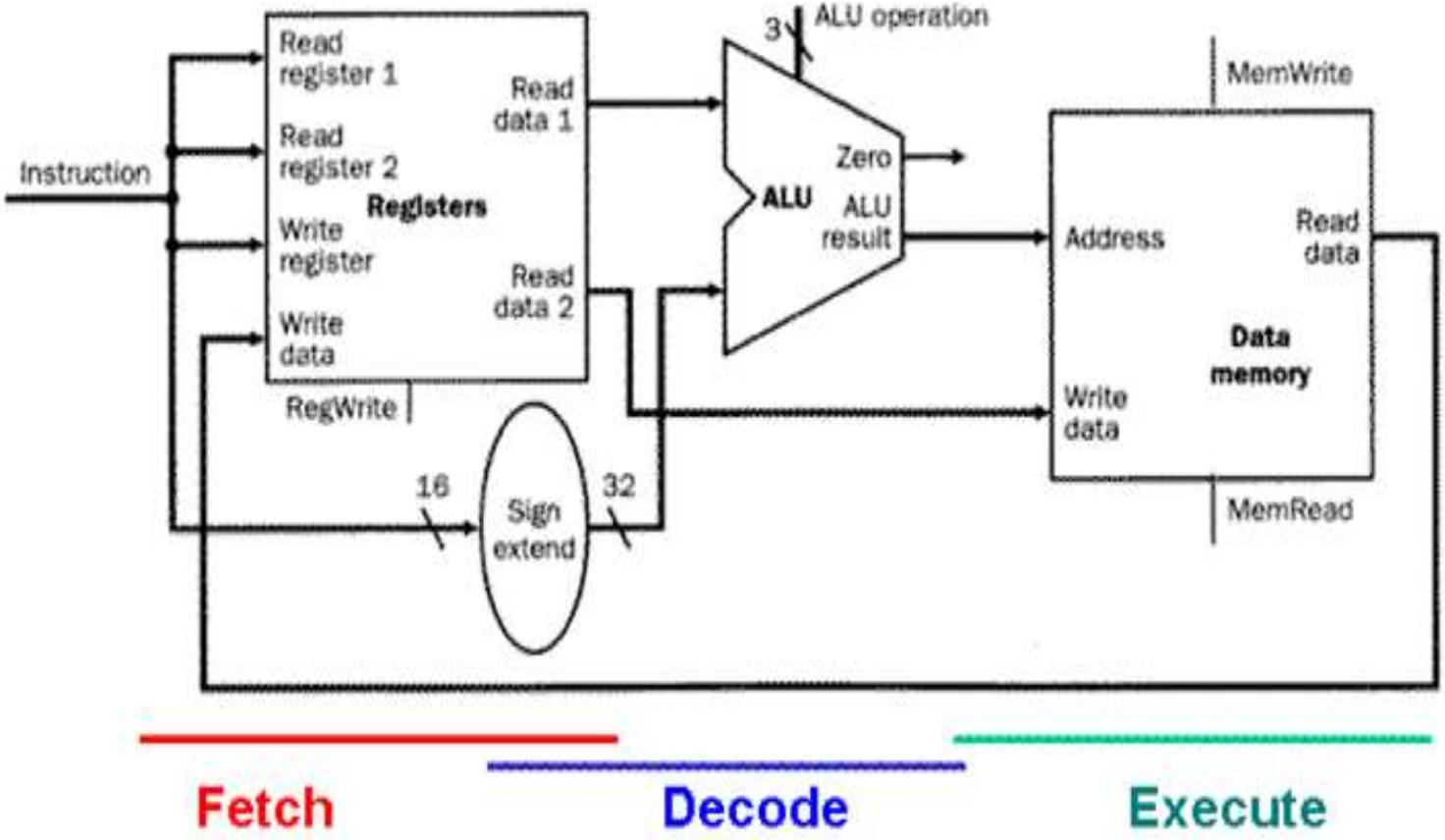
- **Format:** opcode r1, r2, r3



Stage:3 [Data path segment for Branch Instruction]Sign-extend

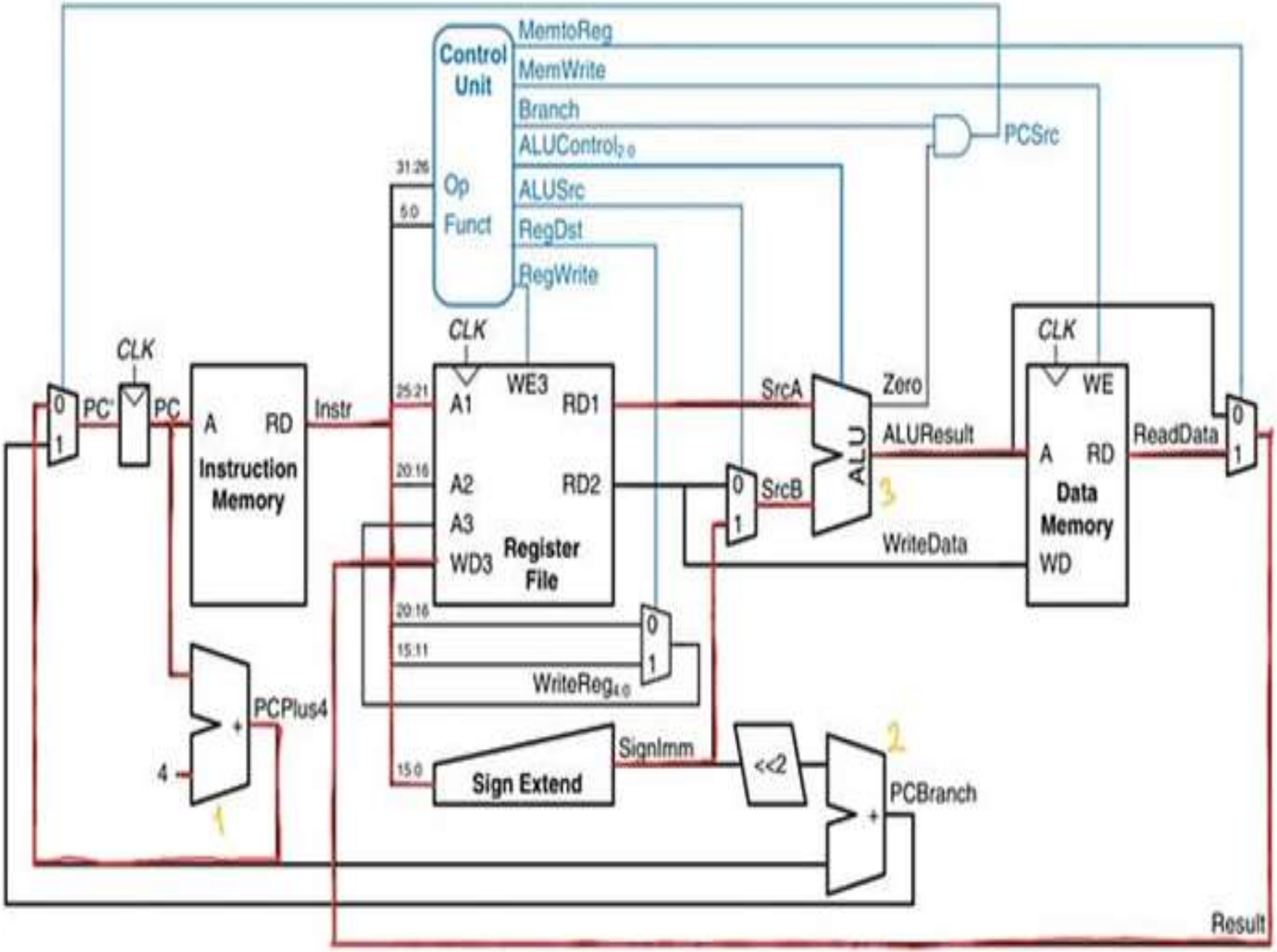


Overall stage diagram



SINGLE-CYCLE PROCESSORS

- In single-cycle processors, each instruction is executed within a single clock cycle.
- This design simplifies the control unit compared to multi-cycle processors, as it eliminates the need for components that manage non-architectural states.


Control Unit
Op
Funct
MemtoReg
MemWrite
Branch
ALUControl2:0
ALUSrc
RegDst
RegWrite
PCSrc
31:26
5:0
CLK
PC'
PC
A
RD
Instruction Memory
Instr
25:21
20:16
A1
A2
A3
WD3
WE3
RD1
RD2
Register File
SrcA
SrcB
ALU
Zero
ALUResult
WriteData
WE
Data Memory
WD
ReadData
Result
20:16
15:11
WriteReg4:0
PCPlus4
4
15:0
Sign Extend
SignImm
<<2
PCBranch
1
2
3

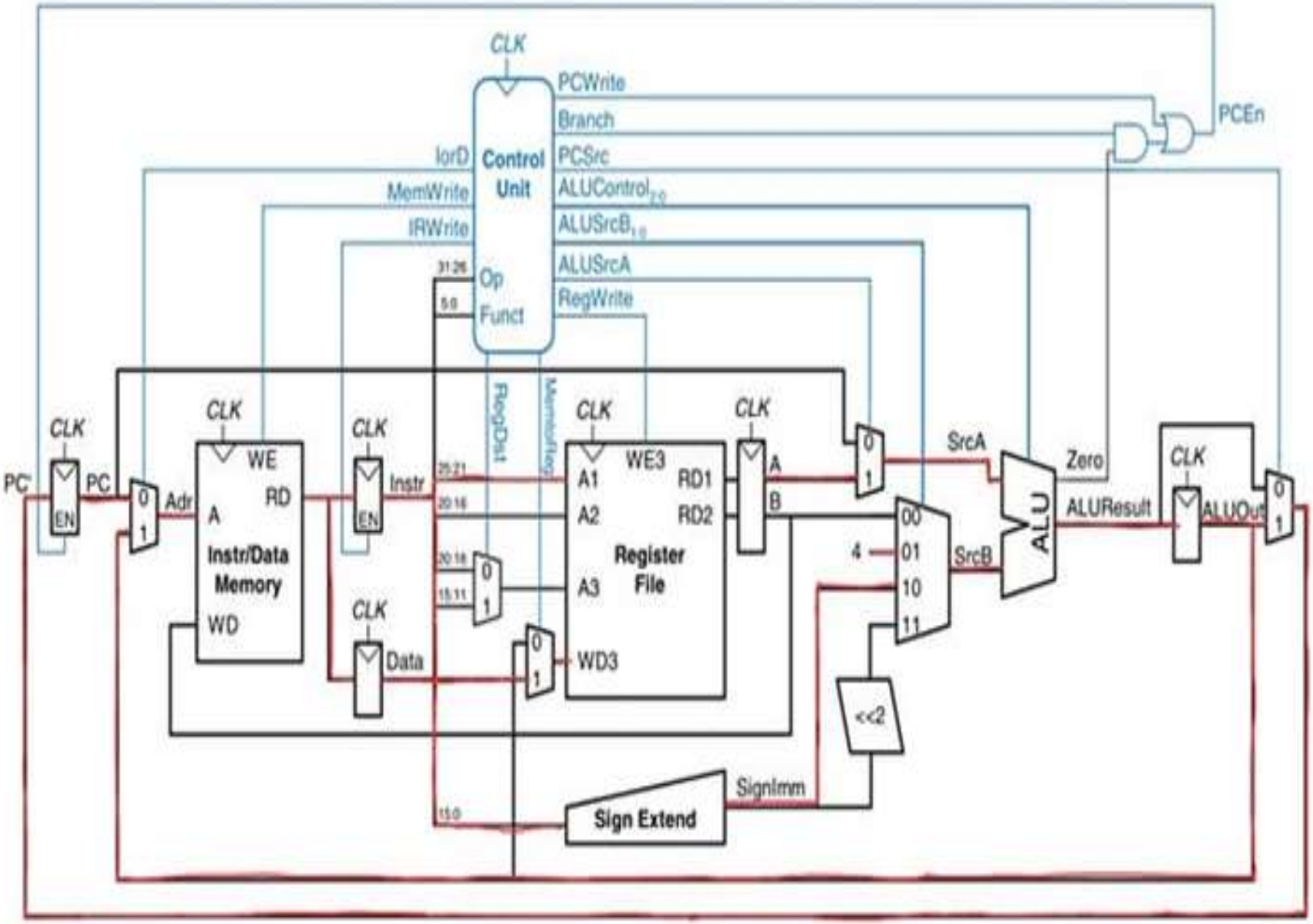
key features of single-cycle processors

1. Fixed Cycle Time
2. Simplicity in Control
3. High Throughput for Simple Instructions

MULTI-CYCLE PROCESSORS

- In a multi-cycle architecture, instructions are executed over a series of shorter clock cycles.
- Steps in Multi-Cycle Execution

1. Instruction Fetch (IF)
2. Instruction Decode (ID)
3. Execution (EX)
4. Memory Access (MEM)
5. Write Back (WB)


CLK
PCWrite
Branch
PCSrc
ALUControl2:0
ALUSrcB1:0
ALUSrcA
RegWrite
Control Unit
Op
Funct
IorD
MemWrite
IRWrite
PCEn
PC'
PC
EN
Adr
WE
RD
A
Instr/Data Memory
WD
Instr
Data
RegDst
MemtoReg
WE3
A1
A2
A3
RD1
RD2
Register File
WD3
SrcA
SrcB
Zero
ALU
ALUResult
ALUOut
00
01
10
11
4
<<2
SignImm
Sign Extend

Features of Multi-Cycle

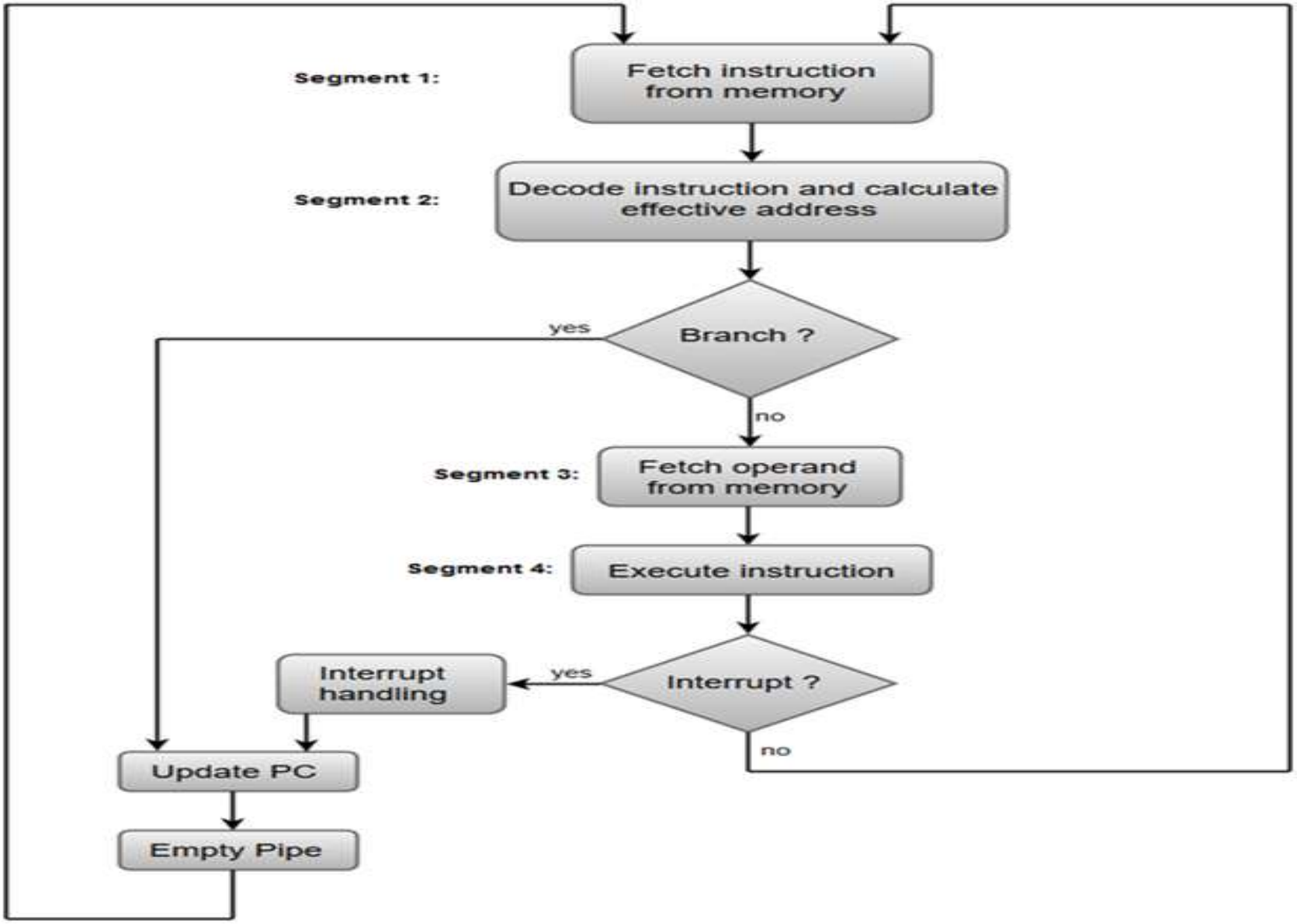
1. Instruction Breakdown
2. Variable Cycle Count
3. Smaller Cycle Time

INSTRUCTION PIPELINING

- An instruction pipeline receives sequential instructions from memory while prior instructions are implemented in other portions.
- Pipeline processing can be seen in both the data and instruction streams.

Steps followed for instruction pipelining

- Fetching the instruction from memory
- 2. Decoding the obtained instruction
- 3. Calculating the effective address
- 4. Fetching the operands from the given memory
- 5. Execution of the instruction
- 6. Storing the result in a proper place


Segment 1:
Fetch instruction from memory
Segment 2:
Decode instruction and calculate effective address
Branch ?
yes
no
Segment 3:
Fetch operand from memory
Segment 4:
Execute instruction
Interrupt ?
yes
no
Interrupt handling
Update PC
Empty Pipe

NOTION OF ILP

- Instruction Level Parallelism (ILP) is used to refer to the architecture in which multiple operations can be performed parallelly in a particular process, with its own set of resources – address space, registers, identifiers, state, and program counters.
- Instruction-level parallelism can also appear explicitly in the instruction set.
- VLIW (Very Long Instruction Word) machines have instructions that can issue multiple operations in parallel.
- Intel IA64 - example of multi processor architecture

DATA AND CONTROL HAZARDS

- Data Hazard is caused when the result (destination) of one instruction becomes the operand (source) of the next instruction.
- Consider two instructions I1 and I2 (I1 being the first).
- Assume I1: INC [4000H]
- Assume I2: MOV BL , [4000H]
- Clearly in I2, BL should get the incremented value of location [4000H].
- But this can only happen once I1 has completely finished execution and also written back the result at [4000H].
- In a multistage pipeline, I2 may reach execution stage before I1 has finished storing the result at location [4000H], and hence get a wrong value of data. This is called data dependency hazard.

- The data hazards are basically of three types:
- 1. RAW -Read After Write-Flow/True Data Dependency
- 2. WAR -Write After Read-Anti-Data dependency
- 3. WAW-Write after Write-Output Data dependency

CONTROL HAZARD

- Pipelining assumes that the program will always flow in a sequential manner.
- Hence, it performs various stages of the forthcoming instructions before-hand, while the current instruction is still being executed.

Example

```
Start:
JMP Down
INC BL
MOV CL, DL
ADD AL, BL
…
…
…
Down: DEC CH
```

JMP Down is a branch instruction.

- After this instruction, program should jump to the location “Down” and continue with DEC CH instruction.

- In a multistage pipeline processor, the sequentially next instructions after JMP Down have already been fetched and decoded. These instructions will now have to be discarded and fetching will begin all over again from DEC CH.
- This will keep several units of the architecture idle for some time.
- This is called a pipeline bubble.
- The problem of branching is solved in higher processors by a method called “Branch Prediction Algorithm”. -Used in Pentium Processors

LIMITATIONS OF ILP

1.The Hardware Model

Register renaming

Branch prediction

Jump prediction

Memory-address alias analysis

2. Limitations on the Window Size and Maximum Issue Count

3.Effects of Realistic Branch and Jump Prediction

4.Limitations on ILP for Realizable Processors